

ResumeFlow — UI Layout & Interaction Report

Lovable implementation specification • Frontend-first MVP • Based strictly on the ResumeFlow core idea

1. Product Definition

ResumeFlow is an existing-resume editing application. A user uploads an existing resume; the system parses and stores the resume content together with the information required to preserve the original template/design; the user edits the extracted content; ResumeFlow produces an updated version using the original template.

Core promise: Edit the content of your existing resume without rebuilding its design.

2. Non-Negotiable Product Rule

The original resume template is locked by default. The application must not automatically redesign, restyle, reorder, resize, recolor, or otherwise modify the user's original resume template. Only content explicitly changed by the user should change in the generated resume.

User edits	System preserves
Name/contact details	Original typography
Summary	Original colors
Skills	Original spacing
Experience	Original margins
Projects	Original section design/order
Education	Original layout
Other resume content	Original visual structure

3. Visual Theme

Use the supplied color reference as the visual foundation. The product should feel warm, editorial, minimal, premium, and professional rather than like a generic blue/purple SaaS dashboard.

Token	Hex	Primary use
Paper	#F2F1ED	Application background
Stone	#DBD5CA	Secondary surfaces, subtle accents
Coral	#EF5848	Primary CTA, active state, important actions
Black	#000000	Primary text, strong controls, navigation
White	#FFFFFF	Cards, inputs, resume document preview

4. Global Application Shell

The app should use a restrained shell. Do not build a large enterprise dashboard. ResumeFlow has one primary job: get the user's resume into an editable state and let them update it.

- **Top bar:** ResumeFlow wordmark on the left; contextual page title or resume name in the center/left; primary action on the right.
- **Navigation:** Keep navigation minimal. The MVP does not need many product areas.
- **Background:** Paper (#F2F1ED).
- **Cards:** White (#FFFFFF) with subtle Stone borders.
- **Primary action:** Coral (#EF5848), with black/white text chosen for contrast.
- **Typography:** modern sans-serif, strong headings, compact labels, comfortable body text.

5. Screen 01 — Landing / Welcome

Purpose: Explain the single problem ResumeFlow solves and move the user to upload their existing resume.

Layout:

- Top-left: ResumeFlow logo/wordmark.
- Main content centered or slightly left-weighted.
- Large headline: “Update your resume without rebuilding it.”
- Supporting text explaining: upload existing resume → edit content → get updated resume.
- Primary Coral button: “Upload Resume”.
- Secondary small note: original design/template is preserved.
- Right-side or lower visual: simplified resume document preview paired with small editor controls.

Interaction: Clicking Upload Resume opens the file picker and then moves to the upload screen.

6. Screen 02 — Resume Upload

Purpose: Let the user provide the existing resume that will become the source document.

- Large centered upload card on the Paper background.
- Dashed/drop zone with a simple document/upload icon.
- Text: “Drop your resume here” and a Browse Files action.
- Show supported file types clearly based on the prototype's supported inputs.
- After selection, replace the empty state with filename, file type, file size, and a remove/change option.
- Primary Coral button: “Analyze Resume”.
- Do not ask the user to choose a new template.

States: Empty, file selected, invalid file, uploading, upload failed.

7. Screen 03 — Parsing / Analysis

This is a short transitional screen. It should make the system feel intentional rather than showing a generic spinner.

- Show the uploaded filename at the top.
- Show a vertical or horizontal progress sequence.
- Step 1: Reading resume.
- Step 2: Detecting sections and content.
- Step 3: Capturing template/layout information.
- Step 4: Preparing editable resume.
- Completed steps use Black/Stone; active step uses Coral.
- When complete, automatically transition to the editor.

8. Screen 04 — Resume Editor (Primary Screen)

This is the most important screen in the product. It should feel like a purpose-built resume editor, not a generic word processor.

Desktop layout:

- Full-height workspace below the top bar.

- Left panel: structured editable resume content.
- Right panel: resume document preview.
- Both panels remain visually distinct.
- The preview uses a White document surface on the Paper application background.

Top editor bar:

- Back/navigation control.
- Resume name.
- Unsaved/Saved status.
- Save Changes button.
- Download button.

Left editor panel:

- Section cards or accordion groups.
- Personal Information.
- Professional Summary.
- Skills.
- Experience.
- Projects.
- Education.
- Other existing sections detected from the uploaded resume.
- Each section should show its extracted values and allow direct editing.

Important: Do not force every resume into the same fixed set of sections. The UI should be able to display sections detected from the uploaded resume while providing sensible common fields.

9. Resume Preview Panel

The preview is not a redesigned ResumeFlow template. It represents the user's original resume.

- White paper-like document centered inside the preview area.
- Preserve the original resume's visual hierarchy in the mock.
- Show page boundaries if the resume spans multiple pages.
- Allow scrolling through the document.
- Optional zoom controls may be provided.
- When the user edits a field, the preview updates to reflect the changed content.
- Do not expose automatic style controls.

10. Editing Interaction Rules

Action	Expected behavior
Edit text	Update local resume state and mark document Unsaved
Add item	Add a new item within the existing resume section
Delete item	Remove only that content item; ask for confirmation when destructive
Save Changes	Persist current frontend state and show Saved
Preview	Focus/expand the resume preview
Download	Trigger the current updated-resume download flow
Navigate away with changes	Show unsaved-changes warning

11. Screen 05 — Saved / Updated State

- After Save Changes, show a small non-blocking confirmation such as “Changes saved”.
- Keep the user on the editor; do not redirect unnecessarily.
- Update the saved timestamp/status.
- Preview remains visible.
- Download remains available.

12. Download Interaction

- Primary download action lives in the editor header.
- Button label: “Download Resume”.
- While generating, show a loading state: “Preparing resume...”
- On success, show a lightweight confirmation.
- The downloaded output represents the edited content using the original template.

13. Error / Empty / Loading States

- No file selected: explain what the user needs to upload.
- Unsupported file: show a clear reason and allow retry.
- Parsing failed: show retry and choose-another-file actions.

- No extracted field: do not invent content; show an empty editable field where appropriate.
- Save failure: preserve the user's local edits and show retry.
- Download failure: keep the editor state intact and allow retry.

14. Responsive Behavior

- Desktop is the primary editing experience.
- On narrower screens, stack the editor and preview vertically.
- Keep the main Save and Download actions easily accessible.
- Do not shrink the resume preview until it becomes unreadable; allow horizontal/vertical scrolling where necessary.
- Upload and landing screens should remain comfortable on mobile browsers.

15. Component Structure for Lovable

Create reusable components rather than putting all UI into one page.

- AppShell
- TopBar
- UploadDropzone
- FileCard
- ParsingProgress
- ResumeEditor
- ResumeSection
- PersonalInfoEditor
- SkillsEditor
- ExperienceEditor
- ProjectEditor
- EducationEditor
- ResumePreview
- PreviewToolbar
- SaveStatus
- DownloadButton
- Toast/Notification

16. Frontend Prototype Data

Use realistic mock data to simulate a parsed technical resume. The mock should contain personal information, summary, skills, at least one experience entry, multiple projects, and education. The important behavior is that editing the structured data immediately updates the preview state.

17. Lovable Implementation Rules

- Build a functional frontend prototype, not static screenshots.
- Use Next.js, TypeScript, and Tailwind CSS.
- Use the exact ResumeFlow color palette defined in this document.

- Keep the resume preview visually paper-like and separate from the application chrome.
- Do not add AI, ATS, JD matching, template selection, or unrelated dashboard features.
- Do not automatically change the user's template/design.
- Use mock services/data for upload, parsing, save, and download until the Spring Boot backend is connected.
- Keep components and state structured so mock services can later be replaced with REST APIs.
- Prioritize the editor and preview experience over secondary pages.

18. Final MVP User Journey

Landing

User understands the product and clicks Upload Resume.

Upload

User selects an existing resume.

Parsing

The application communicates that it is extracting content and preserving the original template.

Editor

User sees extracted resume fields on the left and the original-style resume preview on the right.

Edit

User changes only the information they want to update. Preview reflects those changes.

Save

User saves the updated resume.

Download

User downloads the updated resume with the original template preserved.